

Fastapi

- **FastAPI is a modern, high-performance Python web framework used for building RESTful APIs.**
- It is based on **Starlette** for the web framework and **Pydantic** for data validation.
- It automatically generates interactive API documentation using **OpenAPI (Swagger UI)**.
- Hindi: FastAPI ek modern Python web framework hai jo **API (Application Programming Interface)** banane ke liye use hota hai.

Main Features

- High Performance (ASGI + Starlette ki wajah se)
- Automatic API Documentation (/docs aur /redoc)
- Data Validation using Pydantic
- Easy JWT Authentication support
- Async programming support (async def)

Advantages

FastAPI Advantages and Disadvantages

Advantages

Advantage	Explanation
High Performance	ASGI aur Starlette ki wajah se bahut fast hai.
Automatic API Docs	<code>/docs</code> par Swagger UI automatically milta hai.
Data Validation	Pydantic automatically request data validate karta hai.
Async Support	<code>async/await</code> support karta hai, isliye concurrent requests handle kar sakta hai.
Easy to Learn	Flask jaisa simple syntax hai.
Type Hint Support	Python type hints se IDE auto-completion aur fewer bugs milte hain.
OpenAPI Support	Client SDK aur API testing tools ke saath easily integrate hota hai.

Disadvantages

Disadvantage	Explanation
Smaller Ecosystem	Django ke comparison me packages aur plugins kam hain.
Built-in Admin Panel Nahi	Django ki tarah ready-made admin interface nahi milta.
Authentication Setup Manual Ho Sakta Hai	JWT, OAuth, permissions wagaira khud configure karne padte hain.

Disadvantage	Explanation
Async Concepts Thode Difficult	Beginners ko <code>async</code> , <code>await</code> , event loop samajhne me time lag sakta hai.
Large Monolithic Apps Ke Liye Extra Structure Chahiye	Bahut bade projects me architecture manually organize karna padta hai.
Rapid Changes	Kuch libraries aur versions frequently update hote rehte hain.

Short Interview Summary

Advantages

- High performance
- Automatic Swagger documentation
- Pydantic-based validation
- Async support
- Easy and clean syntax

Disadvantages

- Smaller ecosystem than Django
- No built-in admin panel
- Authentication and permissions require additional setup
- Async programming can be complex for beginners

Simple Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello FastAPI"}

# Run server
uvicorn main:app --reload

# Output:-
{
  "message": "Hello FastAPI"
}
```

- Starlette is a lightweight **ASGI web framework and toolkit** for building asynchronous web applications in Python.
- FastAPI Starlette ke upar built hai.
- Hindi:- Starlette ek lightweight ASGI web framework hai, jiske upar FastAPI bana hua hai.
- It uses Starlette internally for features such as:
 - Routing
 - Request handling
 - Response handling
 - Middleware
 - Background tasks
 - WebSockets
 - Static files
 - Session support

Example

```
from fastapi import FastAPI
from starlette.responses import JSONResponse

app = FastAPI()

@app.get("/")
async def home():
    return JSONResponse({"message": "Hello"})

# Here, JSONResponse comes from Starlette.
```

- Car chalti hai engine ki wajah se, lekin car me extra features hote hain. Usi tarah FastAPI, Starlette ke upar extra features provide karta hai.

ASGI

- ASGI allows a Python web app (FastAPI, Starlette, Django async, etc.) to **communicate** with an asynchronous server (Uvicorn, Hypercorn, Daphne) and handle many requests concurrently using `async/await`.
-
- ASGI (Asynchronous Server Gateway Interface) is a specification that allows Python web applications (FastAPI, Starlette, Django async, etc.) to **communicate** with asynchronous servers such as Uvicorn, Hypercorn, and Daphne. It supports **async/await**, **concurrent requests**, **WebSockets**, and **long-lived connections**.

ASGI Application Signature

- ASGI app ka basic structure:

```
async def app(scope, receive, send):
    pass
```

Parameter	Purpose
scope	Request information (path, method, headers, etc.)
receive	Incoming messages receive karta hai
send	Response bhejta hai

ASGI is Used?

- Handle Multiple requests concurrently
- Support WebSockets
- Support Long-lived connections
- Perform asynchronous operations using async/await

ASGI server

ASGI Server	Use
Uvicorn	FastAPI में सबसे ज्यादा इस्तेमाल होता है
Hypercorn	HTTP/2 और QUIC जैसी features support करता है
Daphne	Django Channels के साथ popular है
Granian	High-performance modern ASGI server
Gunicorn + Uvicorn Workers	Production deployment में बहुत common combination

uvicorn

- Uvicorn is an ASGI server used to run FastAPI applications.
- Uvicorn is a lightweight, high-performance ASGI server used to run FastAPI applications.
- Uvicorn एक ASGI server है जो FastAPI application को run करने के लिए इस्तेमाल किया जाता है।

What does Uvicorn do?

- HTTP requests को receive करता है
- ASGI protocol के अनुसार FastAPI से communicate करता है
- Response वापस client को भेजता है
- Async requests को efficiently handle करता है
- WebSocket support भी देता है

Note

- ASGI = एक protocol / specification है (rules/नियम)
- Uvicorn = उन नियमों पर काम करने वाला server

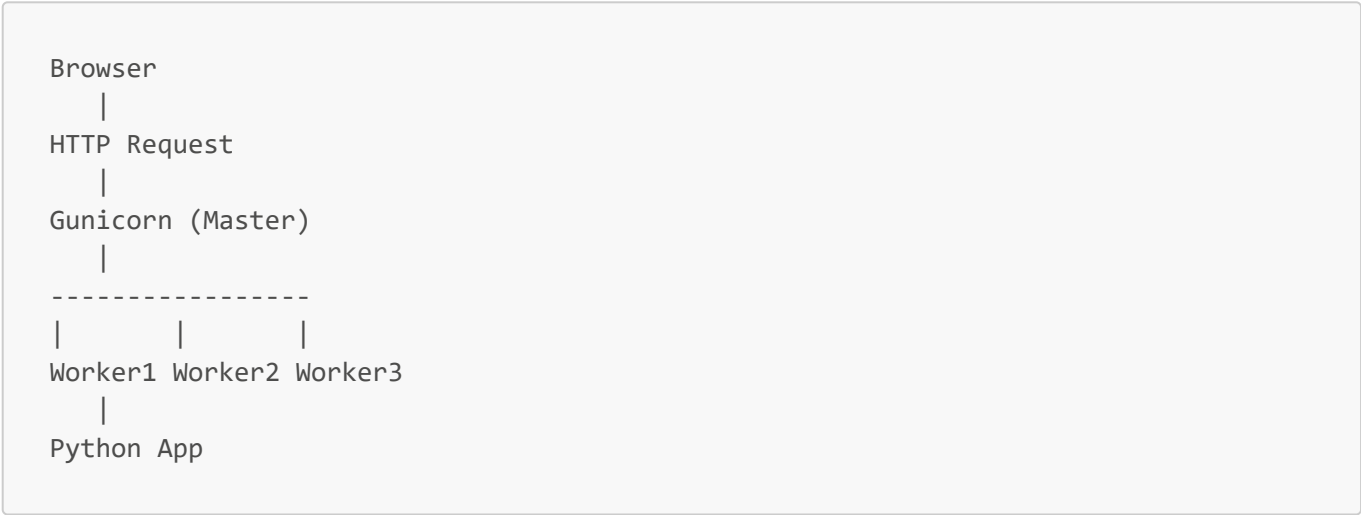
- FastAPI = application/ASGI-compatible framework

Gunicorn

- Gunicorn (Green Unicorn) is a Python WSGI HTTP server used to run Python web applications such as Django and Flask in production environments.
- It works by creating a master process and multiple worker processes to handle concurrent HTTP requests efficiently. For ASGI frameworks like FastAPI, Gunicorn is commonly used together with Uvicorn workers.
- Hindi:- यह एक Python WSGI HTTP Server है जिसका उपयोग Python web applications (जैसे Django, Flask) को production में चलाने के लिए किया जाता है।

Gunicorn कैसे काम करता है?

- Gunicorn एक master process बनाता है और उसके अंदर कई worker processes चलाता है।



- इससे एक साथ कई requests handle हो सकती हैं।

Gunicorn और Uvicorn में Difference

Feature	Gunicorn	Uvicorn
Interface	WSGI	ASGI
Frameworks	Django, Flask	FastAPI, Starlette
Async support	नहीं (native)	हाँ
Use case	Traditional Python web apps	Modern async apps

FastAPI में Gunicorn क्यों?

- Production में जब ज़्यादा traffic हो, तब केवल एक Uvicorn process कम पड़ सकता है।
- इसलिए हम Gunicorn को **process manager** की तरह उपयोग करते हैं।
- Process Manager का मतलब है ऐसा software जो application के processes को manage करता है।

```
gunicorn -w 4 -k uvicorn.workers.UvicornWorker app.main:app
```

How do you install FastAPI?

GET

How do you create a GET endpoint in FastAPI?

- In FastAPI, a GET endpoint is created using the **@app.get() decorator**. The decorator defines the URL path, and the function below it handles the incoming GET request and returns the response, usually as a Python dictionary which FastAPI automatically converts to JSON.
- Hindi:- FastAPI में GET endpoint बनाने के लिए @app.get() decorator का उपयोग किया जाता है।

Type of Get Parameters

Simple get

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello FastAPI"}

# Output:-
{
  "message": "Hello FastAPI"
}
```

GET Endpoint with Path Parameter

- URL के अंदर value आती है।

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{user_id}") # path parameter
def get_user(user_id: int):
    return {"user_id": user_id}

# request:- GET /users/10
```

GET Endpoint with Query Parameter

- URL के ? के बाद value आती है।

```

from fastapi import FastAPI

app = FastAPI()

@app.get("/items")
def get_items(limit: int = 10):    # query parameter
    return {"limit": limit}

# request:-
GET /users?limit=5

```

- Multiple Query Parameters

```

@app.get("/search")
def search(name: str, city: str):
    return {"name": name, "city": city}

# request:-
GET /search?name=Mohit&city=Delhi

```

Multiple Query Parameters

```

@app.get("/search")
def search(name: str, city: str):
    return {"name": name, "city": city}

# Request:- GET /search?name=Mohit&city=Delhi

```

Optional Query Parameter

```

from typing import Optional

@app.get("/products")
def get_products(category: Optional[str] = None):
    return {"category": category}

# Request
GET /products
GET /products?category=electronics

```

Path + Query Together

```
@app.get("/users/{user_id}")
def get_user(user_id: int, active: bool = True):
    return {
        "user_id": user_id,
        "active": active
    }

#request
GET /users/10?active=false
```

Query Parameter Validation

```
from fastapi import Query

@app.get("/items")
def get_items(limit: int = Query(10, ge=1, le=100)):    # ge=1 → minimum 1, le=100
    → maximum 100
    return {"limit": limit}
```

GET with Header Parameter

- Client के headers से data पढ़ता है।

```
from fastapi import Header

@app.get("/profile")
def get_profile(user_agent: str = Header()):
    return {"user_agent": user_agent}
```

GET with Cookie Parameter

- Cookies से value प्राप्त करता है।

```
from fastapi import Cookie

@app.get("/session")
def get_session(session_id: str = Cookie(None)):
    return {"session_id": session_id}
```

GET with Dependency Injection

- Authentication, database session आदि के लिए बहुत उपयोगी।


```
from fastapi import Depends

def common_params():
    return {"page": 1}

@app.get("/items")
def get_items(params: dict = Depends(common_params)):
    return params
```

GET with Response Model

- Response को validate और document करता है।

```
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str

@app.get("/user", response_model=User)
def get_user():
    return {"id": 1, "name": "Mohit"}
```

GET with Status Code

```
from fastapi import status

@app.get("/health", status_code=status.HTTP_200_OK)
def health():
    return {"status": "OK"}
```

Async GET Endpoint

```
@app.get("/data")
async def get_data():
    return {"message": "async endpoint"}
```